12

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

İlk Spring Boot Pod Manifestomuzu Yazma (YAML Anatomisi)

Writing Your First Spring Boot Pod Manifest (YAML Anatomy)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes'e bir dilekçe yazıyoruz! Bu resmi dilekçenin 4 kutsal bölümü vardır: 1) Hangi kanun maddesi? (`apiVersion`), 2) Ne istiyorsun? (`kind: Pod`), 3) Bu şeyin adı ne olsun? (`metadata: name`), 4) Özellikleri neler olsun? (`spec: containers`). İşte bu 4 parçayı doğru yazarsanız K8s dilekçenizi saniyeler içinde işleme koyar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Writing a Kubernetes YAML is filing an official petition to the cluster government! Every petition requires 4 sacred fields: 1) What law version? (`apiVersion`), 2) What resource type? (`kind: Pod`), 3) What is its name and tag? (`metadata`), 4) What are the exact specifications? (`spec`). Get those four right, and K8s grants your wish instantly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bir K8s manifestosunun yapısı: `apiVersion` (kaynağın API sürümü; örn: Pod için `v1`, Deployment için `apps/v1`), `kind` (oluşturulacak nesne tipi), `metadata` (name, namespace, labels, annotations gibi tanımlayıcı veriler), `spec` (kaynağın istenen teknik durumu; örn: imaj adı, portlar, ortam değişkenleri). `kubectl apply -f manifest.yaml` komutu verildiğinde API Server manifestoyu doğrular, etcd'ye kaydeder ve Kubelet konteyneri çeker.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The anatomy of every K8s resource: 1) `apiVersion` specifies the API endpoint group, 2) `kind` declares the schema type, 3) `metadata` houses names, namespaces, and queryable labels, 4) `spec` defines desired characteristics. Submitting via `kubectl apply -f` triggers schema validation in the API Server before etcd persistence.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# spring-boot-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: spring-coffee-pod
  labels:
    app: coffee-service
    env: dev
spec:
  containers:
    - name: coffee-container
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
          name: http
      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"

# Pod'u çalıştır ve loglarını izle:
# kubectl apply -f spring-boot-pod.yaml
# kubectl logs -f spring-coffee-pod
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

YAML dosyalarında TAB tuşu KESİNLİKLE yasaktır! Hizalama için sadece boşluk (space - genelde 2 boşluk) kullanılmalıdır, aksi takdirde YAML ayrıştırma hatası alırsınız.

🇬🇧 WATCH OUT

YAML syntax forbids TAB characters! Use strictly 2 spaces for indentation, or the API Server parser will reject the manifest with syntax errors.

🎯 Özet Çıkarımlar

- Her K8s kaynağı apiVersion, kind, metadata ve spec alanlarını içerir.
- Labels (etiketler) nesneleri gruplamak ve filtrelemek için hayati önem taşır.
- Pod logları `kubectl logs -f <pod-name>` ile anlık olarak takip edilir.

🎯 Key Takeaways

- Every K8s object schema shares apiVersion, kind, metadata, and spec.
- Labels are key-value pairs enabling dynamic grouping, querying, and routing.
- Inspect application stdout/stderr streams directly using `kubectl logs`.